

Object-Based Databases

Seminar Presentation

Presented by: Misna Jebin K

Introduction to Object-Based Databases

- Complex application domains require handling of complex data types.
- These data types can include:
 - Nested record structures (records within records)
 - Multivalued attributes (one attribute having multiple values)
 - Inheritance (properties passed down from parent to child classes)
- Traditional relational databases struggle with these.
- Object-Relational Data Model helps by:
 - Extending relational data models
 - Supporting complex data types and object-oriented features

Object-Relational Database Systems

- These systems combine the strengths of:
 - Relational databases (tables, rows, columns, SQL queries)
 - Object-oriented programming (objects, inheritance, encapsulation)
- Why it's useful:
 - Provides a migration path for users who want object-oriented features
 - No need for a complete system redesign
 - Can work with both relational and object-based data smoothly

Approaches to Implement Object-Based Databases

There are two main approaches:

① Object-Oriented Database System:

- A database that natively supports object-oriented data types.
- Data can be accessed directly using an object-oriented programming language.
- Uses the type system of the programming language.

② Object-Relational Mapping (ORM):

- A technique to convert data between:
 - Programming language's native data types
 - Relational database format
- Automatically handles data conversions in both directions.
- Tools: Hibernate (Java), SQLAlchemy (Python), Entity Framework (.NET)

Summary

- Object-based databases handle complex, real-world data more effectively.
- They blend the advantages of relational databases and object-oriented programming.
- Two implementation methods:
 - ① Build a native object-oriented database system.
 - ② Use an ORM framework for seamless data conversion between objects and tables.
- Ideal for applications requiring complex relationships, inheritance, and nested structures.